SOVEREIGN MASTER BUILD SPECIFICATION

Canonical handoff for Claude Code and Codex

Status: Active master specification
Audience: Claude Code, Codex, founder, reviewers, future builders
Purpose: Consolidate the product vision, technical architecture, current repository state, Build 1 scope, security model, integrations, real-time UX, communities, self-created apps, and Idea-to-Impact coordination into one source of truth.

---

0. Instructions to builders

Read this file before making product or architecture changes.

Priority order

1. Preserve the kernel authority and security invariants.
2. Preserve the canonical ontology.
3. Ship the Build 1 vertical slice.
4. Do not expand unrelated money, social, or autonomous systems.
5. Prefer adapters and declarative composition over new infrastructure.
6. Every meaningful mutation must have authorization and an event.
7. Every feature must be testable with a real user and a real artifact.

Builder rules

- Do not restart or throw away the existing kernel.
- Do not create a second authorization system.
- Do not let models directly mutate the database, kernel, secrets, or external services.
- Do not trust browser-supplied actor, owner, community, sandbox, or represented-human IDs.
- Do not put private content, prompts, personal data, or secrets on a public blockchain.

- Do not make money, token, wallet, payment, or external side effects part of the default Build 1 flow.
- Do not create overlapping concepts for Person, Bot, Community, World, Sandbox, Session, Capability, Grant, Artifact, and Event.
- Do not add a feature without defining its data boundary, capability requirements, event behavior, and tests.
- During security review, do not add unrelated features.

Change protocol

For every task:

1. State the exact problem.
2. Identify the ontology objects involved.
3. Identify the data flow and privacy classification.
4. Identify required capabilities and approvals.
5. Define kernel events.
6. Add positive and negative tests.
7. Run relevant tests, typecheck, lint, and build where practical.
8. Report assumptions, limitations, and files changed.

---

1. Product thesis

The product

> **Sovereign is a self-serve platform where people, bots, and communities create shared sandboxes, collaborate on real-world goals, and turn ideas into attributable, verifiable outcomes.**

Sovereign provides the plumbing:

- Portable identity.
- Bot identity independent of AI provider.
- Community membership and governance.
- Sandboxed collaboration.

- Capability grants and revocation.
- AI/provider adapters.
- Real-time presence and work status.
- Artifacts, contribution lineage, and evidence.
- Self-created UX, rooms, workflows, and mini-apps.
- Social and tool integrations.
- Privacy-preserving integrity anchoring.

Users decide what they create together.

The human promise

> **Build together. See what is happening. Know what happened. Make the next real-world move.**

The long-term purpose

Sovereign helps transform collaborative intellectual effort into coordinated real-world action while preserving shared trust in:

- Who contributed.
- Who authorized an action.
- What context was used.
- What was proposed versus completed.
- What evidence supports a claim.
- What actually changed.
- What can be reused by another community.

What Sovereign is not

It is not primarily:

- A chatbot.
- An avatar game.
- A generic enterprise AI workspace.
- A blockchain coin.
- A social feed.

- A metaverse.
- A payment rail.
- A generic agent framework.
- A replacement for every tool users already have.

Those may be interfaces, capabilities, or integrations. The durable center is identity, authority, community, sandbox, collaboration, artifact, evidence, and portability.

---

2. Current repository reality

The active repository snapshot already contains meaningful infrastructure. Do not rebuild it from scratch.

Known current areas from the repository snapshot:

Plain Text

```
supabase/migrations/      memory kernel, money, payments, authority/actions/Wor
lib/kernel/client.ts      typed client for kernel_api
lib/kernel/server.ts      kernel as signed-in user
lib/chain/                DEV-SOLVE/Solana, genesis, vaults, transfers, RPC gat
lib/money/settlement.ts   settlement worker and reconciliation
lib/router/               payment/venue routing
lib/money/payment.ts      payment worker
lib/ops/                  operating loop, workers, verifiers, learning
lib/world/                graph model, Horizon, deixis, presentation, response
lib/sandbox/              Arcadia browser kernel, PGlite, snapshot boot/reset
app/world/                World shell, globe/Cesium, Horizon, lenses, context d
app/                      Next.js app shell
scripts/                  proof-of-life and operational scripts
tests/                    kernel, money, payments, router, chain, World, browse
```

Known stack:

Plain Text

```
Next.js App Router
Tailwind CSS
Supabase Auth + Postgres
PGlite browser kernel/sandbox parity
Solana/DEV-SOLVE chain modules
```

Current strengths

- Identity registration.
- Agent creation with no authority by default.
- Grants and revocation.
- Provenance and represented human.
- Append-only kernel events.
- Hash-chain verification.
- Evidence bases.
- Gated `kernel_api` reads/writes.
- World graph and presentation model.
- Manipulable globe with activity dots.
- Browser sandbox foundation.
- Existing capability/tool slice.
- Significant tests and build documentation.

Current risks

- The repository expresses several products at once.
- Community is not yet a fully explicit first-class object.
- Sandbox may still be more internal implementation than self-serve product.
- Bot/provider abstraction needs to be visibly independent.
- Social and external integration boundaries need to be formalized.
- Money/chain systems can distract from the platform proof.
- Live deployment previously had Vercel Deployment Protection blocking app access.

Existing money/chain rule

Keep money, payments, settlement, routing, and chain work tested and documented, but feature-gate it. Do not delete it and do not put it in the Build 1 user path.

Plain Text

```
ENABLE_MONEY=false
ENABLE_PAYMENTS=false
```

```
ENABLE_EXTERNAL_FINANCIAL_ACTIONS=false
```

3. Canonical ontology

Use only these canonical objects in Build 1.

Plain Text

```
Person
Bot
Community
World
Sandbox
Session
Capability
Grant
Delegation
Intent
Goal
Milestone
Task
Commitment
Resource
Artifact
Contribution
Outcome
Provider
Runtime
ExternalConnection
KernelEvent
Anchor
UXExtension
```

Definitions

Object	Definition
Person	Human Sovereign identity
Bot	Persistent AI Sovereign identity independent of provider
Community	People and bots governed by shared rules

World	Visible project/collaboration space
Sandbox	Technical and data boundary inside a World
Session	Human/bot collaboration process
Capability	Named action that may be authorized
Grant	Scoped permission for a capability
Delegation	Authority passed from one identity to another
Intent	Desired change expressed by a person/community
Goal	Measurable target derived from an Intent
Milestone	Verifiable checkpoint toward a Goal
Task	Bounded unit of work with owner, scope, risk, and evidence
Commitment	Deliberate promise by a person, organization, or bot owner
Resource	Time, skill, data, equipment, money later, space, materials, or permission
Artifact	Produced work
Contribution	Attributable participation in an artifact, decision, or task
Outcome	What actually happened, distinct from the plan
Provider	Claude, OpenAI, Muse, local model, or future AI service
Runtime	Implementation powering a Bot
ExternalConnection	Linked service such as Discord, Slack, GitHub, Drive, or Notion
KernelEvent	Authoritative record of identity, authority, action, result, and evidence
Anchor	External cryptographic commitment to a verified event checkpoint
UXExtension	User-created declarative room, view, workflow, or mini-app

Core relationships

Plain Text

```
Person owns Bot
Person joins Community
Community contains People and Bots
Community owns or shares Worlds
World contains Sandboxes
Sandbox contains Sessions and Artifacts
Bot participates in Sessions
Grant authorizes Person or Bot inside a scope
Session produces Artifacts and Contributions
Intent creates Goals
Goal contains Milestones
Milestone contains Tasks
Task may have Commitments and Resources
Outcome reports the result of a Goal
Provider powers Runtime
Runtime powers Bot
ExternalConnection maps an outside service to a Sovereign identity
KernelEvent records authority and result
Anchor commits to a verified event checkpoint
UXExtension renders within a Sandbox/World scope
```

Terminology rules

- Product language may use “bot.” Internal code may use `agent` only where inherited from existing APIs, but do not create a second conceptual identity.
- World is a user-visible project space.
- Sandbox is the technical/data boundary within a World.
- Community can contain multiple Worlds.
- Provider is not Bot identity.
- Kernel event is not raw content storage.
- Public proof is not public private data.

---

4. Final Build 1 scope

Build 1 mission

Build a thin full-thesis version: differentiated enough to test Sovereign’ s actual idea, small enough for one human and a focused engineering loop to secure.

Included

- Person Sovereign ID.
- Bot Sovereign ID.
- One private Community with optional public profile.
- One private World/Sandbox.
- Two human participants.
- Two bots.
- Two AI provider adapters: Claude plus OpenAI or a private/local test runtime.
- One social bridge: Discord or Slack, not both.
- One document artifact and one code or visual artifact, selected by pilot.
- Low-risk capability grants.
- Human approval.
- Real-time human/bot presence.
- Agent execution statuses.
- Activity/provenance panel.
- Revocation.
- Export.
- One fork/remix template.
- Kernel hash-chain verification.
- Optional operator/public Merkle anchor.
- One Idea-to-Impact goal/milestone/task loop.
- One user-created declarative UX extension.

Excluded

- Payments.
- Tokens.

- Consumer wallets.
- Financial transfers.
- Sensitive personal, medical, legal, or financial data.
- Autonomous irreversible actions.
- Public bot marketplace.
- Public access to private sandboxes.
- Cross-community federation.
- Zero-knowledge proofs.
- Arbitrary tool marketplace.
- Full public blockchain explorer.
- General-purpose workflow builder.

Risk envelope

The first pilot uses low-risk project data and no irreversible external side effects.

---

5. Golden user journey

Plain Text

```
Visit public community profile
  -> sign in/create Person ID
  -> create or join private AI Build Room
  -> create Bot or choose template
  -> connect Claude/OpenAI/private runtime
  -> review provider and data policy
  -> invite second person
  -> add second Bot
  -> grant scoped capabilities
  -> define Intent and Goal
  -> accept first Milestone
  -> collaborate in real time
  -> create document/code/visual Artifact
  -> review Contributions and Evidence
  -> approve selected version
  -> publish intentionally selected result
  -> optionally share through selected social bridge
  -> optionally anchor checkpoint
```

```
  -> revoke or export
  -> fork/remix room/template
```

Default AI Build Room

Plain Text

```
Community: private, optional public profile
World/Sandbox: one project room
Person A: owner/steward
Person B: contributor
Bot A: Planner, read + propose
Bot B: Critic, read-only
Provider: Claude
Second provider: optional portability test
Artifact: Markdown project brief by default
External actions: disabled
Social posting: approval required
Blockchain anchor: optional/operator-only initially
```

The user enters a project name and goal. The user does not configure the ontology.

---

6. Idea-to-Impact system

Core loop

Plain Text

```
Spark       someone expresses an intention
Frame       room clarifies problem and desired change
Gather      people, bots, evidence, and resources assemble
Design      group creates possible approaches
Commit      participants choose roles, tasks, and boundaries
Execute     authorized work happens
Verify      results are checked against evidence and milestones
Reflect     group records what worked and failed
Reuse       room, artifacts, bots, and methods are forked/shared
```

Task classes

Digital

Draft, analyze, design, summarize, create internal artifact.

Human

Interview, visit site, review design, approve budget, attend meeting.

External

Send message, schedule appointment, create GitHub issue, submit proposal.

Physical

Install, deliver, measure, repair, manufacture.

Irreversible

Spend money, sign contract, delete, change security, publish consequential statement.

Sovereign coordinates all classes, but does not claim that a Bot physically completed a task without evidence.

State distinction

Plain Text

```
Proposed
Approved
In progress
Reported complete
Verified
Measured outcome
```

AI coordinator loop

Plain Text

```
Observe state
  -> find ready/blocked tasks
  -> propose next action
  -> check authority/risk
  -> request human commitment or approval
  -> execute or assign
  -> collect evidence
```

```
  -> update milestone
  -> surface exceptions
```

The coordinator may suggest; it does not gain authority by suggesting.

Trust/evidence ladder

Plain Text

```
Level 0: self-reported
Level 1: system-observed
Level 2: artifact-backed
Level 3: witnessed by authorized participant
Level 4: independently corroborated
Level 5: externally verified or measured
```

Do not reduce trust to one universal score. Trust is contextual by task type, community, evidence, recency, and capability.

Disagreement is a first-class state:

Plain Text

```
question
counterclaim
dispute
evidence_request
review
resolution
unresolved
```

---

7. Real-time Build Rooms

Core definition

> **A Build Room is a living workspace where people and their agents visibly create something together in real time.**

It contains:

- Shared canvas.
- Human/bot presence.

- Live work status.
- Goals and tasks.
- Artifacts and versions.
- Permissions and approvals.
- Activity/provenance.
- World/globe connection.

Universal room shell

Plain Text

```
┌──────────────────────────────────────────────────────────────────┐
| SOVEREIGN  World / Community / Room     Live  Share Publish |
├──────────────────┬──────────────────────────────┬────────────────┤
| Participants  |                              | Activity /     |
| Bots          |       DOMAIN CANVAS          | Proof          |
| Artifacts     |                              | Live status     |
| Goals         | CRM / 3D / Solar / Ad        | What changed   |
| Permissions   | / Code / Avatar              | Approvals      |
├──────────────────┴──────────────────────────────┴────────────────┤
| Ask · assign · create · approve · publish                  |
└──────────────────────────────────────────────────────────────────┘
```

Invariants

Top bar:

- Sovereign mark.
- World/community/room path.
- Live state.
- Participant count.
- Privacy state.
- Share/publish.
- Optional anchor status.

Left rail:

- People.
- Bots.

- Goals.
- Tasks.
- Artifacts.
- Permissions.

Center:

- Domain canvas.

Right:

- Live activity.
- Agent status.
- Proposals.
- Approvals.
- Recent changes.
- Evidence.

Bottom:

- One command bar for ask, assign, create, approve, and publish.

Presence states

Plain Text

```
idle
thinking
reading_context
planning
proposing
waiting_for_approval
executing
writing_artifact
blocked
completed
failed
paused
revoked
```

Use real runtime events. Do not fake “thinking.”

Real-time categories

Ephemeral

Presence, cursor, selection, typing, camera, viewport.

Use Supabase Realtime or equivalent.

Durable

Messages, proposals, approvals, artifact changes, goals, permissions, kernel events.

Use Postgres/kernel events.

Streaming execution

Started, context loaded, tool proposed, awaiting approval, executing, writing, completed, failed.

Stream to room; persist meaningful results and final event.

Room event envelope

Plain Text

```
interface RoomEvent {
  id: string;
  roomId: string;
  actorId: string;
  actorType: "person" | "bot" | "system";
  type:
    | "presence.changed"
    | "session.message"
    | "agent.status"
    | "proposal.created"
    | "approval.requested"
    | "approval.decided"
    | "artifact.changed"
    | "goal.updated"
    | "permission.changed"
    | "anchor.updated";
  payload: unknown;
  visibility: "private" | "community" | "public";
  createdAt: string;
  sequence?: number;
}
```

8. Domain canvases

The shell remains stable. The canvas is a plug-in.

Plain Text

```
interface BuildRoomCanvas {
  canvasId: string;
  domain: string;
  roomId: string;
  mount(container: HTMLElement, context: CanvasContext): CanvasController;
  serialize(): Promise<ArtifactPayload>;
  applyOperation(operation: CanvasOperation): void;
  getSelections(): Selection[];
  dispose(): void;
}
```

Potential canvases:

Plain Text

```
CRMCanvas
ProductDesignCanvas
House3DCanvas
SolarCanvas
AdCanvas
AvatarCanvas
DocumentCanvas
CodeCanvas
```

First pilot chooses one flagship canvas. Solar is especially strong because it connects globe, location, real-world data, multiple bots, visual work, calculations, and an external proposal artifact.

---

9. Globe/World model

Use three scales:

Plain Text

```
Globe = where public activity and communities exist
World = what a community/project is doing
Room = how people and bots build it now
```

The current manipulable globe is the World layer, not the room canvas.

Public globe layers

```
Intent layer       what people want to change
Community layer    who is organizing around it
Project layer      what is being built
Resource layer     what is available publicly
Outcome layer      what has actually happened
```

Location privacy

```
private
approximate region
city-level
public point
```

Default to private or approximate region. Exact public points require explicit publication.

Public activity card

```
Westside Solar Pilot
Status: active
Community: Westside Energy Lab
Goal: reduce energy costs
Milestone: 3 of 5
Public outcome: 18% modeled reduction
Evidence: system-observed + reviewed
Join / view public artifact
```

The globe should feel like a living map of what people and bots are building, not a surveillance map.

---

10. UX Kernel and self-extending app

Core idea

> **Users can create new rooms, views, workflows, dashboards, and mini-apps inside Sovereign without developer deployment.**

The UX evolves while the kernel remains stable.

Internal versus external

Internal composition without outside permissions

- Layouts.
- Views.
- Forms.
- Dashboards.
- Canvases.
- Internal workflows.
- Bot configuration.
- Artifact templates.
- Internal triggers.
- Calculations over approved sandbox data.

External capabilities

- Social posting.
- GitHub/Drive access.
- Email.
- External API calls.
- Public publication.
- Money.
- Cross-community access.
- Security changes.

External actions require declared capabilities and usually human approval.

UX extension flow

Plain Text

```
Natural-language intent
  -> generated draft UX manifest
  -> preview
  -> human edits
  -> schema validation
  -> capability review
  -> private activation
  -> community sharing
  -> public template
  -> fork/remix
```

UX manifest

Plain Text

```
interface UXExtensionManifest {
  id: string;
  version: string;
  name: string;
  description?: string;
  ownerId: string;
  communityId?: string;
  sandboxId: string;
  visibility: "private" | "community" | "public";
  entryView: string;
  views: UXViewDefinition[];
  commands: UXCommandDefinition[];
  workflows: UXWorkflowDefinition[];
  requiredCapabilities: string[];
  dataBindings: DataBinding[];
  theme: ThemeDefinition;
  status: "draft" | "active" | "paused" | "archived";
}
```

Declarative views

Users compose safe primitives:

Plain Text

```
View, Panel, Canvas, Field, Card, Table, Chart, Timeline, Graph, Command, Workf
```

No arbitrary database access.

Data bindings

Bindings are scoped to current room/community/sandbox resources:

Plain Text

```
room.goal
room.members
room.bots
room.tasks
room.artifacts
room.activity
artifact.versions
sandbox.files
proposal.metrics
site.location
```

Commands

All mutations are named commands with schema, risk, capability, scope, and approval mode.

Plain Text

```
artifact.create_version
proposal.request_review
task.assign_member
task.mark_blocked
room.create_bot
template.save
view.publish
```

Workflows

Internal event-driven workflows can run without external permissions:

Plain Text

```

```

```
proposal version created
  -> calculate output
  -> ask Critic Bot to review
  -> create review task
  -> notify room
  -> wait for human approval
```

External steps pause for a capability and approval.

Formulas

Support safe calculations with no arbitrary JavaScript, network, filesystem, mutation, or secrets.

Examples:

- Solar output.
- Lead score.
- Project progress.
- Material cost.
- Ad budget.
- Energy estimate.

Code extensions

Declarative UX is default. Future code extensions require sandboxed runtime, declared capability manifest, network allowlist, storage scope, and resource limits.

User-created extensions cannot:

- Change kernel policy.
- Grant themselves authority.
- Read arbitrary communities.
- Read service keys.
- Publish externally without capability.

---

11. Integrations

Integration strategy

Sovereign owns:

- Identity.
- Ownership.
- Membership.
- Delegation.
- Capability grants.
- Sandbox policy.
- Sessions.
- Artifacts.
- Provenance.
- Revocation.
- Portability.

Adapters provide:

- AI intelligence.
- Execution.
- Social reach.
- Storage.
- Search.
- External tools.
- Hosting.

Integration contract

Every integration must define:

1. Sovereign object mapping.
2. Capabilities exposed.
3. Data entering/leaving.
4. Kernel events.
5. Revocation behavior.
6. Failure/retry behavior.

7. Privacy classification.

AI providers

Claude

Primary first runtime.

OpenAI or private/local runtime

Second runtime to demonstrate Bot identity is not provider lock-in.

Muse

Experimental adapter; provider-specific behavior stays isolated.

Plain Text

```
interface BotRuntime {
  providerId: string;
  runtimeId: string;
  connect(input: ConnectInput): Promise<RuntimeConnection>;
  disconnect(connectionId: string): Promise<void>;
  generate(input: RuntimeInput): Promise<RuntimeOutput>;
  stream?(input: RuntimeInput): AsyncIterable<RuntimeEvent>;
}
```

Portable:

- Bot ID.
- Owner.
- Profile.
- Memberships.
- Capability policy.
- Approved memory export.
- Artifact/event history.

Not guaranteed portable:

- Identical answers.
- Identical personality.
- Identical hidden memory behavior.

- Identical tool semantics.

Sandboxes

Use existing sandbox foundation and plug in specialized runtimes where needed. Do not compete with E2B-like providers on microVM infrastructure.

Tool broker

Plain Text

```
interface ToolBroker {
  listAvailableTools(input: ToolListInput): Promise<Tool[]>;
  proposeCall(input: ToolCallProposal): Promise<ProposedAction>;
  executeApproved(actionId: string): Promise<ActionResult>;
}
```

Social bridge

Build one bridge first:

- Discord if the first audience is a community.
- Slack if the first audience is a work team.

Capabilities:

Plain Text

```
social.read_selected_channel
social.post_approved_artifact_link
social.create_session_link
```

Requirements:

- Explicit account linking.
- Selected channel only.
- No full-history import by default.
- Human approval for outbound posts.
- Idempotency keys.
- Bot attribution.
- Immediate disconnect/revocation.

- Kernel events.

GitHub/files

Sovereign does not replace Git history or document storage. It adds:

- Person/bot attribution.
- Community/sandbox origin.
- Capability used.
- Human approval.
- Context references.
- Artifact lineage.

Build 1 GitHub capabilities, if selected:

```
Plain Text

github.read_repository
github.create_branch
github.create_draft_pr
```

No automatic merge or destructive repository operations.

---

12. Privacy and security

Trust zones

```
Plain Text

Browser: untrusted
Application API: authenticated request handling
Kernel API/database: authority source of truth
Provider/runtime: untrusted model output
Sandbox: bounded execution/data boundary
External integration: untrusted third party
Blockchain: public integrity witness only
```

Non-negotiable rule

> **The model never directly controls the database, kernel, secrets, or external systems.**

Provider policy

Each Bot/Sandbox has a provider policy:

```
Plain Text

interface ProviderPolicy {
  providerId: string;
  mayReceiveApprovedProjectData: boolean;
  mayReceivePersonalData: boolean;
  mayReceiveSensitiveData: boolean;
  retentionMode: "unknown" | "provider_policy" | "zero_retention" | "local_only
  region?: string;
  allowExternalTools: boolean;
  requireHumanApprovalForExternalActions: boolean;
}
```

Show context before sending:

```
Plain Text

Provider
Data being sent
Data excluded
Retention status
External tools
Sensitive-data policy
```

Data classification

```
Plain Text

public
community
project
private
sensitive
secret
```

Defaults:

- Private Community.

- Private World/Sandbox.
- Private Session.
- Approved bot context only.
- No public indexing.
- Explicit publishing.

Content versus integrity

Private content

Messages, prompts, model outputs, files, memories, personal data, private membership, sensitive evidence, credentials.

Integrity metadata

Pseudonymous actor, event type, scope, capability, timestamp, status, content commitment, previous event hash, evidence reference.

Secrets

- Never in browser bundles.
- Never in prompts, logs, events, or artifacts.
- Managed secret storage.
- Narrow injection into runtimes.
- Rotation and revocation.

Prompt injection

Treat documents, emails, web pages, memories, community messages, and tool results as untrusted data.

Context labels:

```
SOVEREIGN POLICY: authoritative
HUMAN REQUEST: current intent
WORLD DATA: untrusted content
TOOL RESULT: untrusted external output
```

Permissions

Plain Text

```
Authenticated caller?
Correct identity?
Community member?
Sandbox member?
Active grant?
Scope matches?
Approval required?
Approval valid?
Risk/resource limits pass?
```

Security tests

- Cross-user sandbox read fails.
- Cross-community read fails.
- Bot starts with no authority.
- Ungranted tool call fails.
- Revoked/expired grant fails.
- Service key cannot impersonate human.
- Browser actor IDs ignored.
- Prompt injection cannot change authority.
- Path traversal rejected.
- Unauthorized network/private IP blocked.
- Tool output cannot become system instruction.
- External sends require approval.
- Duplicate actions are idempotent.
- Secrets absent from browser/model/logs/events/artifacts.
- Hidden content excluded from retrieval.
- Export excludes secrets.
- Backups restore successfully.

13. Blockchain

Role

Blockchain is an optional integrity witness, not private storage, identity replacement, payment requirement, or product center.

Build 1 anchor flow

Plain Text

```
Verified kernel events
  -> canonical serialization
  -> Merkle checkpoint
  -> optional chain transaction
  -> anchor registry
  -> inclusion verification
```

On-chain only:

- Root/checkpoint hash.
- Network/program identifier.
- Protocol version.
- Timestamp/slot.

Never on-chain:

- Conversation text.
- Prompts/model outputs.
- Personal information.
- Membership lists.
- Private files.
- Secrets.
- OAuth tokens.

No wallet/token required. Chain outage must not stop ordinary work.

User-facing explanation:

> “Sovereign stores private activity off-chain and can anchor a cryptographic fingerprint so unauthorized history changes can be detected. Private content is not published to the

> blockchain.”

Existing money/chain code stays feature-gated. Separate anchor module from settlement/payment.

---

14. Brand and visual system

Brand promise

> **Build together. See what is happening. Know what happened.**

Brand architecture

Plain Text

```
Sovereign World
  -> Community
    -> Build Room
      -> Domain Canvas
        -> Artifacts, agents, people, activity, proof
```

Universal room grammar

- Sovereign identity rings.
- Human/bot distinction.
- Community constellation.
- Live presence states.
- Activity/proof rail.
- Artifact/version timeline.
- World/globe breadcrumb.
- Green verified state.
- Same command bar.
- Same privacy and permission language.

Semantic palette

Plain Text

```
Sovereign green = identity, granted permission, healthy live state
Deep space = World/canvas depth
Blue = information/reading/context
Violet = bot proposal/creative possibility
Amber = human attention/approval required
Red = blocked/failed/revoked
```

Use motion to represent real state, not decoration.

Globe role

The globe is a map of public Worlds, communities, activity, resources, and outcomes. It is not the room canvas and not a surveillance surface.

---

15. Self-creation loop

Natural-language creation

Home prompt:

> **What are you trying to make real?**

Input:

> “Help our neighborhood reduce energy costs.”

Suggested scaffold:

Plain Text

```
Community: Energy Lab
Room: Neighborhood Solar Pilot
Roles: coordinator, researcher, solar designer, resident reviewer
Bots: data researcher, solar planner, finance bot
Milestones: gather data, validate, design, partner, measure
First action: review data sources
```

The user can accept, edit, or reject suggestions.

Progressive disclosure

Plain Text

```
Intent
  -> room template
  -> people/bots
  -> plain-language capabilities
  -> first milestone
  -> next action
  -> artifact/outcome
  -> reusable template
  -> fork/remix
```

The app should always surface:

- Current goal.
- Current milestone.
- Next best action.
- Who is needed.
- What is blocked.
- Why a bot suggested something.
- What evidence remains uncertain.

---

16. Implementation sequence

Days 1–3: staging truth

- Resolve Vercel Deployment Protection for public pilot staging.
- Confirm dedicated staging Supabase.
- Expose `kernel_api`; keep `kernel` private.
- Verify real JWT claims.
- Run build/typecheck/lint/unit/browser/World tests.
- Run critical authorization tests against real Postgres.

Days 4–8: golden path

- Person registration.
- Community creation.

- World/Sandbox.
- Bot creation.
- Claude connection.
- Second human invite.
- Three capability grants.
- Session/messages.
- Markdown artifact.
- Activity/provenance.
- Revocation.
- Export.

Days 9–12: differentiation

- Second provider adapter.
- Public Community profile.
- Explicit artifact publication.
- Fork/remix template.
- Provider policy/context preview.
- One UXExtension generated from natural language.

Days 13–16: social bridge

- Select Discord or Slack.
- Link one channel.
- Read approved context.
- Post approved artifact link.
- Idempotency and revocation.

Days 17–20: impact and integrity

- Intent/Goal/Milestone/Task subset.
- Next-action recommendation.
- Evidence level.

- Kernel chain verification.
- Optional Merkle checkpoint/anchor.
- Provider leakage, prompt-injection, sandbox, and secret tests.
- Backup/restore.

Days 21–30: real pilot

- Recruit 5–15 users.
- Run one real project.
- Observe without over-explaining.
- Fix repeated blockers only.
- Ask whether they will use it next week.
- Decide what earns Build 2.

---

17. Claude Code operating prompt

Plain Text

```
You are implementing Sovereign from SOVEREIGN-MASTER-BUILD-SPEC.md.

Read the repository’s DIRECTIVES.md, VISION-CANON.md, BUILD-BOOK.md, SECURITY-I

Implement only the explicitly requested task. Reuse kernel_api identity/grant/e

For every mutation:
- resolve identity server-side;
- enforce community/sandbox/capability scope;
- use the kernel authority path;
- record an event;
- add negative authorization tests;
- document data flow and privacy behavior.

Before finishing:
- run relevant tests;
- run typecheck/lint/build where practical;
- report changed files;
- report security assumptions;
- report known limitations;
- do not broaden scope.
```

Suggested Claude Code tasks

1. Community schema/CRUD and events.
2. Community membership and roles.
3. Sandbox membership/policy.
4. Bot/provider runtime abstraction.
5. Tool broker and capability checks.
6. Session/proposal/approval/artifact flow.
7. Activity/provenance UI.
8. Provider policy/context preview.
9. UX manifest registry and declarative renderer.
10. Natural-language room scaffold.
11. One domain canvas.
12. One social bridge.
13. Intent/Goal/Milestone/Task subset.
14. Merkle checkpoint and anchor service.
15. Verification and privacy tests.

---

18. Codex operating prompt

Plain Text

```
Review the current repository against SOVEREIGN-MASTER-BUILD-SPEC.md.

Do not add features during this review. Find defects and ontology drift, ranked

Focus on:
- actor/owner spoofing;
- grant and revocation bypasses;
- cross-community/sandbox leaks;
- provider context over-sharing;
- service-key exposure;
- tool broker bypasses;
- prompt-injection paths;
- external integration scope errors;
- public links exposing private content;
```

```
- secrets in logs/events/artifacts;
- unsafe UX extension commands;
- formulas or code extensions escaping the sandbox;
- blockchain payload containing PII/content;
- anchor verification errors;
- missing kernel events;
- false claims in UI/documentation;
- missing real-Postgres concurrency tests.

Return:
1. Critical findings.
2. High findings.
3. Medium/low findings.
4. Missing tests.
5. Minimal fixes.
6. Risks that require human decision.
Do not rewrite unrelated modules.
```

19. Definition of done

Product

- First-time user completes setup in 15 minutes or less.
- Two humans and two bots collaborate in one private Build Room.
- One useful artifact is produced.
- One second provider can power a Bot without changing Bot identity.
- One selected social bridge can share an approved artifact link.
- One UXExtension can be created and previewed without deployment.
- One room can be forked/remixed without copying private data.
- One Intent/Goal/Milestone/Task path reaches a meaningful state.

Security

- No critical authorization findings.
- New Bots start with no authority.
- Revocation blocks future actions immediately.
- All tool calls pass the broker.

- External actions require approval.
- Provider receives only approved context.
- Secrets are absent from browser, prompts, logs, events, and artifacts.
- Code/formula extensions cannot escape their sandbox.

Privacy

- Private content is off-chain.
- Membership is private by default.
- Provider policy is visible.
- Hidden content is excluded from retrieval.
- Deletion language is accurate.
- Public artifacts expose only intentionally published data.
- Location is private/approximate by default.

Integrity

- Kernel tampering fails verification.
- Correct event inclusion verifies against an anchor.
- Incorrect proof fails.
- Chain outage does not stop ordinary work.
- No wallet/token is required.

Demand

- At least one community returns for four weeks.
- Members create real work, not only demos.
- Someone invites another participant.
- Someone asks to reuse, fork, or extend a room/template/bot.
- A steward would pay, sponsor, or seriously commit to continuation.

---

20. Final doctrine

> **Build the smallest version that still demonstrates why Sovereign is different.**

The foundation is:

```
Portable identity
+ community participation
+ private sandbox
+ real-time room
+ provider choice
+ capability control
+ attributable artifacts
+ Idea-to-Impact coordination
+ self-created UX
+ social reach
+ optional integrity anchor
```

No money, token, sensitive data, or irreversible autonomy in the first pilot.

> **The kernel is the foundation. The community is the user. The room is the living interface. The artifact is the proof. The outcome is the value. The integrations are the reach. The blockchain is the witness—not the product.**